



R3.04

Résumé Pattern Stratégie

Adam Adnane

Fatima Abouda

Evan Arranga

Geoffrey Rigal

Table des matières

R3.04	1
Résumé Pattern Stratégie	1
Sommaire :	2
L'intention métier du pattern	2
Diagramme de classes (GoF).....	3
Description du diagramme générique	3
Classes participantes	3
Exemple du contexte métier	4
Contexte : Calculateur de prix dans un site e-commerce	4
Mauvaise solution	4
Problèmes	4
Bonne solution	4
Solidarté du pattern	5

L'intention métier du pattern

Définir une famille d'algorithmes, les encapsuler séparément et les rendre interchangeables.

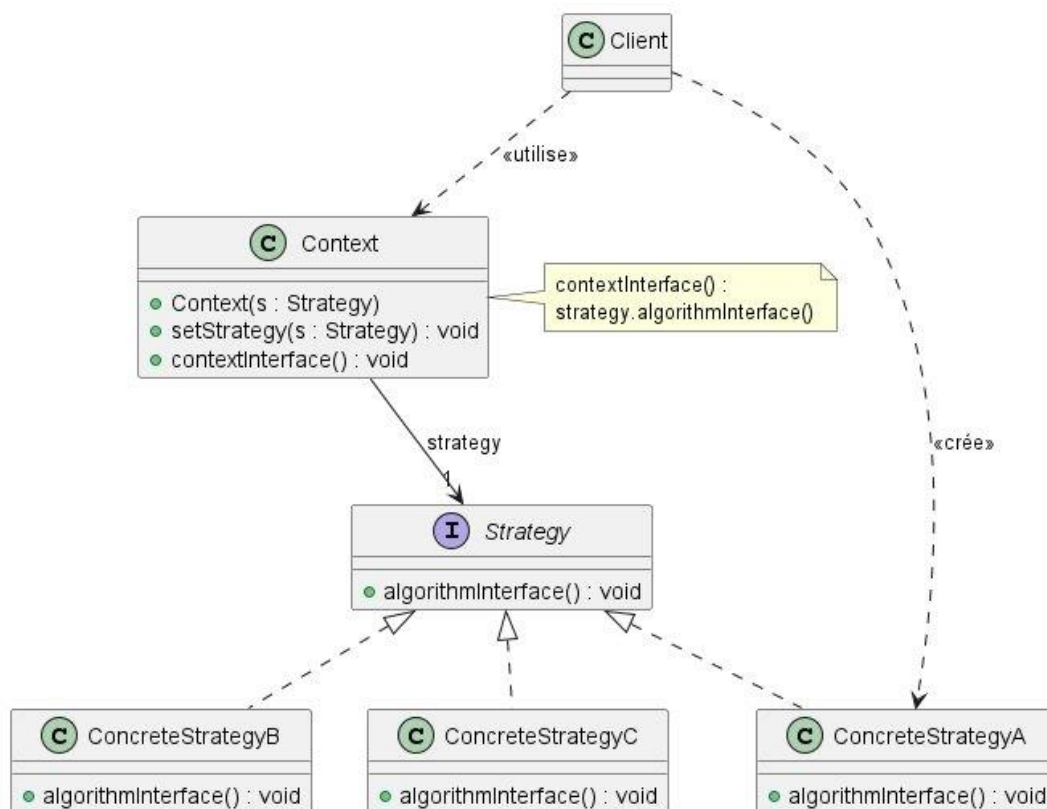
Le pattern Stratégie permet de modifier l'algorithme indépendamment de l'objet qui l'utilise.

Pour faire plus simple :

- L'algorithme ou le comportement est externalisé dans une stratégie
- La classe principale ne connaît pas l'implémentation, seulement l'interface
- On peut changer le comportement dynamiquement sans toucher au code du contexte
- Le code devient flexible, extensible et maintenable

Diagramme générique du pattern Stratégie

Diagramme de classes (GoF)



Description du diagramme générique

- **Context** : Détient une référence vers une stratégie. Délègue l'exécution du comportement à cette stratégie
- **Strategy** : Interface définissant l'algorithme abstrait.

- ConcreteStrategy : Implémentation de l'algorithme. Chaque classe représente une variante du comportement.

Classes participantes

Le pattern Stratégie repose sur trois types de classes, chacun ayant un rôle bien défini dans la structure du pattern.

Ces classes collaborent pour rendre un comportement interchangeable, extensible, et modifiable dynamiquement.

CLASSE DU PATTERN	DEFINITION	ROLE
CONTEXT	Utilise une stratégie pour exécuter un comportement	Détient une référence vers l'interface Strategy. Délègue l'algorithme à la stratégie.
STRATEGY	Interface définissant l'algorithme abstrait.	Spécifie le contrat que toutes les stratégies concrètes doivent respecter.
CONCRETSTRATEGY	Implémentations réelles de l'algorithme.	Chaque classe représente une variante du comportement. Le contexte peut les interchanger dynamiquement.

Exemple du contexte métier

Contexte : Calculateur de prix dans un site e-commerce

Un site e-commerce doit appliquer différents types de réductions sur un prix :

- Réduction de 10 %
- Réduction de 20 %
- Aucune réduction

Mauvaise solution

Le calculateur contient une méthode :

```
double calculerPrix(double prix, String typeReduction) {
    if (typeReduction.equals("10")) return prix * 0.9;
    if (typeReduction.equals("20")) return prix * 0.8;
    return prix;
}
```

Problèmes

- Le code viole SRP : plusieurs comportements dans une seule méthode.
- Le code viole OCP : ajouter une réduction ce qui modifier la méthode.
- Le code viole DIP : dépendance à des valeurs (strings).
- Le code devient difficile à maintenir et à tester.

Bonne solution

On applique le pattern Stratégie

Interface :

```
interface StrategiePrix {  
    double calculerPrix(double prix);  
}
```

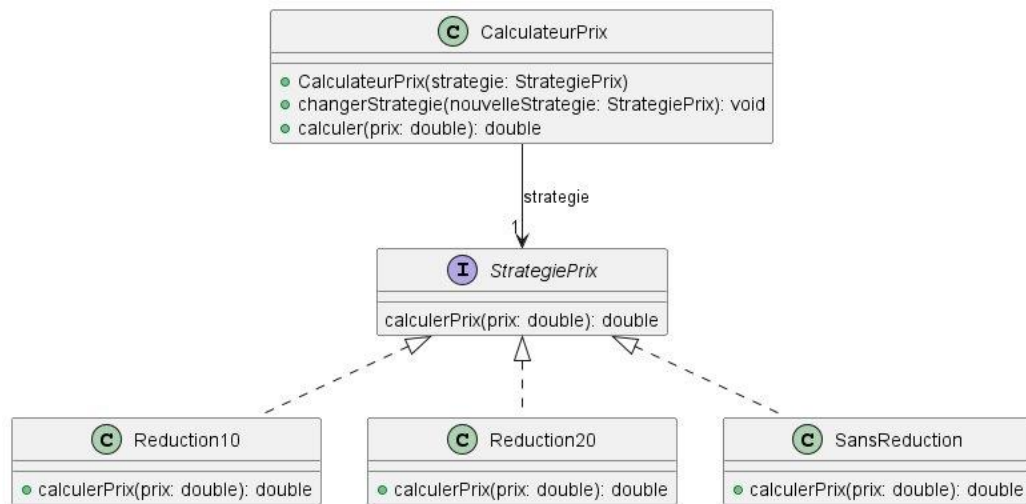
Stratégies concrètes

- Reduction10
- Reduction20
- SansReduction

Chacune implémente son propre calcul

Contexte : CalculateurPrix

```
class CalculateurPrix {  
    private StrategiePrix strategie;  
    public CalculateurPrix(StrategiePrix strategie) {  
        this.strategie = strategie;  
    }  
    public void changerStrategie(StrategiePrix nouvelle) {  
        this.strategie = nouvelle;  
    }  
    public double calculer(double prix) {  
        return strategie.calculerPrix(prix);  
    }  
}
```



Solidarité du pattern

Le pattern Stratégie respecte pleinement les principes SOLID :

- SRP : chaque stratégie a une seule responsabilité (un algorithme).
- OCP : on ajoute une nouvelle stratégie sans modifier le contexte.
- LSP : toutes les stratégies peuvent remplacer l'interface sans casser le code.
- ISP : l'interface Strategy est petite et spécifique.
- DIP : le contexte dépend d'une abstraction (Strategy), pas d'une implémentation.